

1. La Machine à États (FSM) : Le "Cerveau" de TCP

Contrairement à l'IP ou l'ICMP qui traitent chaque paquet comme s'il était le premier, TCP est un protocole **à état**.

Le problème sans FSM : Si je reçois un paquet avec le flag ACK

, comment savoir si c'est la suite d'une connexion légitime ou une erreur ?

- Si je n'ai jamais vu de SYN avant, c'est une erreur.
- Si j'ai envoyé un SYN-ACK juste avant, c'est la fin du handshake.

La solution : Chaque connexion est suivie dans notre table `tcb_table`

Chaque entrée a un state

- **LISTEN** : État par défaut de notre serveur qui attend que quelqu'un frappe à la porte.
- **SYN_RECEIVED** : On a reçu une demande (SYN) et on a répondu "OK" (SYN-ACK). On attend maintenant la confirmation
- **ESTABLISHED** : La confirmation (ACK) est arrivée. La ligne est ouverte, on peut s'échanger des données.

Pourquoi c'est important ? Cela permet de rejeter les paquets qui n'ont rien à faire là. Si quelqu'un envoie des données sans avoir fait le SYN d'abord, notre FSM voit que l'état est CLOSED et ignore ou rejette le paquet.

2. Le rôle crucial du paquet RST (Reset)

Le flag RST est le bouton "Arrêt d'urgence" de TCP. Nous l'avons implémenté dans la fonction `tcp_send_reset`. On l'utilise dans deux cas critiques :

1. **Refus de connexion** : Si quelqu'un essaie de se connecter mais que notre table de connexions est pleine (protection contre le SYN Flood).
2. **Paquet invalide** : Si on reçoit un paquet qui ne correspond à aucune connexion ouverte ou qui arrive au mauvais moment.

En envoyant un RST, on dit à l'autre machine : *"Arrête tout, je ne te connais pas ou ce que tu m'envoies n'a pas de sens"*.

3. Focus Sécurité : Le "Stealth Scanning" (Scan Furtif)

L'outil `nmap -sS` (SYN Scan) utilise exactement ce que nous venons de coder.

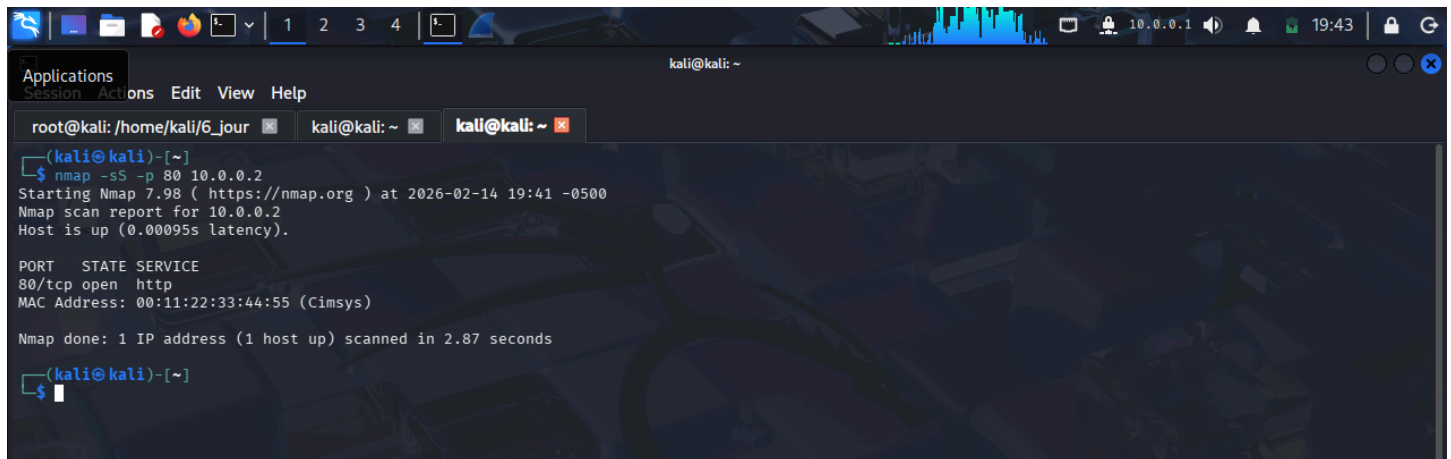
Comment nmap marche ?

1. `nmap` envoie un SYN

2. Notre stack répond SYN-ACK

- À cet instant précis,
- nmap
- a sa réponse : **"Le port est OUVERT"**. Il n'a pas besoin d'aller plus loin.

3. Pour être "furtif", au lieu d'envoyer le ACKfinal qui établirait la connexion, nmap envoie un RST



```
Applications
Session Actions Edit View Help

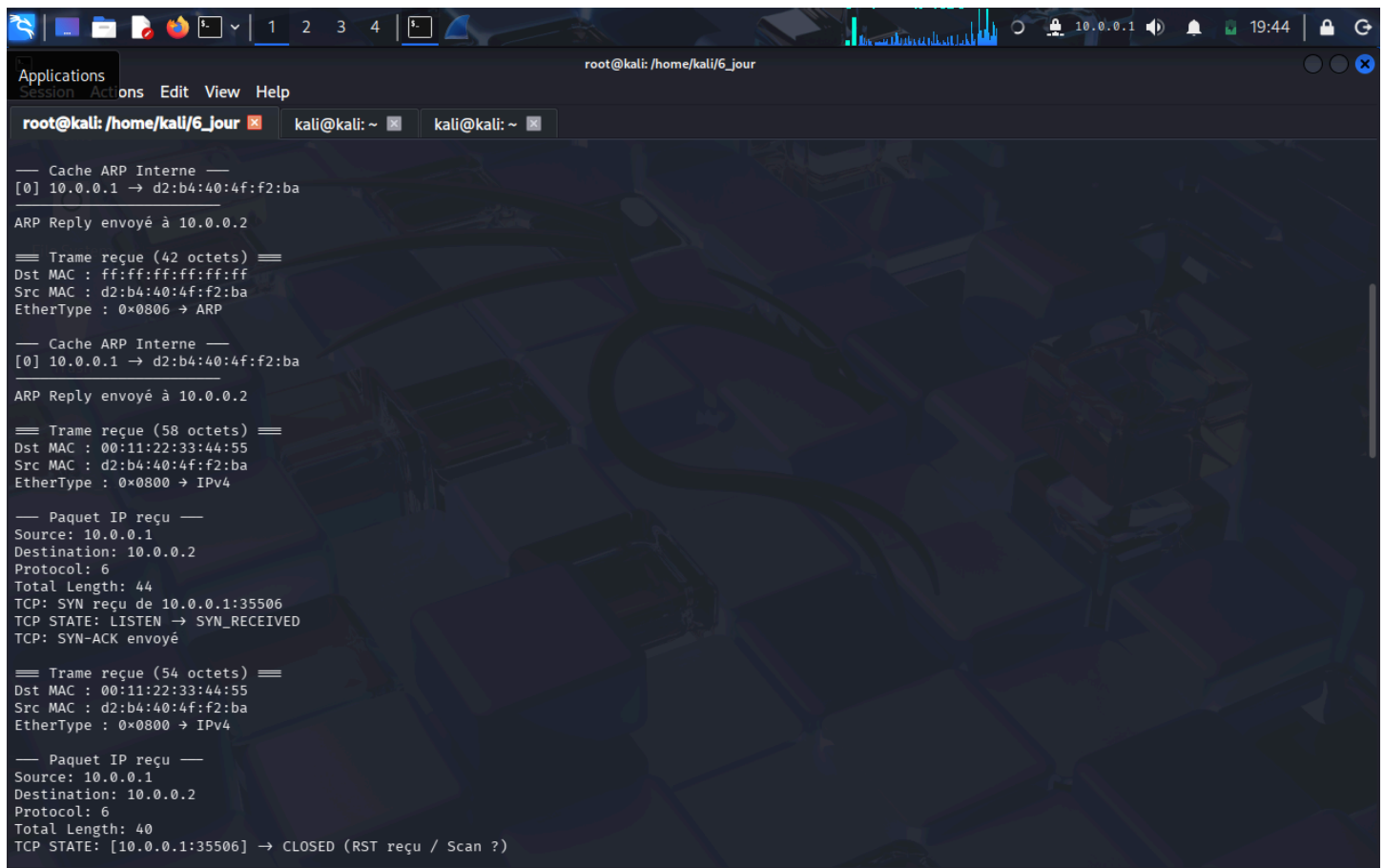
root@kali: /home/kali/6_jour kali@kali: ~ kali@kali: ~

(kali@kali)~]
$ nmap -sS -p 80 10.0.0.2
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-14 19:41 -0500
Nmap scan report for 10.0.0.2
Host is up (0.00095s latency).

PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 00:11:22:33:44:55 (Cimsys)

Nmap done: 1 IP address (1 host up) scanned in 2.87 seconds

(kali@kali)~]
$
```



```
Applications
Session Actions Edit View Help

root@kali: /home/kali/6_jour kali@kali: ~ kali@kali: ~

--- Cache ARP Interne ---
[0] 10.0.0.1 → d2:b4:40:4f:f2:ba

ARP Reply envoyé à 10.0.0.2

== Trame reçue (42 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : d2:b4:40:4f:f2:ba
EtherType : 0x0806 → ARP

--- Cache ARP Interne ---
[0] 10.0.0.1 → d2:b4:40:4f:f2:ba

ARP Reply envoyé à 10.0.0.2

== Trame reçue (58 octets) ==
Dst MAC : 00:11:22:33:44:55
Src MAC : d2:b4:40:4f:f2:ba
EtherType : 0x0800 → IPv4

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 44
TCP: SYN reçu de 10.0.0.1:35506
TCP STATE: LISTEN → SYN_RECEIVED
TCP: SYN-ACK envoyé

== Trame reçue (54 octets) ==
Dst MAC : 00:11:22:33:44:55
Src MAC : d2:b4:40:4f:f2:ba
EtherType : 0x0800 → IPv4

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 40
TCP STATE: [10.0.0.1:35506] → CLOSED (RST reçu / Scan ?)
```